Lispy |

(Task 1 of 9) In this tutorial, we will learn about **lambda expressions**, which are expressions that create functions.

The following program illustrates how to create a function without giving it a name and then call it immediately.

Lispy [Run ▶]

```
((lambda (n)
   (+ n 1))
 2)
```

Scala 3

```
((n : Int) => n + 1)(2))
```

The function is created by a lambda expression. This function increases it parameter by one.

Lispy [Run ▶]

```
(lambda (n)
  (+ n 1))
```

Scala 3

```
(n : Int) => n + 1
```

Any feedback regarding these statements? Feel free to skip this question. 1

(You skipped the question.) 2

Lispy |

(Task 2 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(deffun (f x)
  (lambda (y) (+ x y)))
(defvar x 0)
((f 2) 1)
```

JavaScript

```
function f(x) {
  return function (y) {
    return x + y;
  };
}
let x = 0;
console.log(f(2)(1));
```

3 4

Lispy |

You predicted the output correctly 🎉

This program is essentially the same as

Lispy [Run ▶]

```
(deffun (f x)
  (deffun (fun y)
    (+ x y))
  fun)
(defvar x 0)
((f 2) 1)
```

Python

```
def f(x):
    def fun(y):
        return x + y
    return fun
x = 0
print(f(2)(1))
```

Click here to run this program in the Stacker.

Lispy |

(Task 3 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(deffun (bar y)
  (lambda (x)
    (+ x y)))
(defvar f (bar 2))
(defvar g (bar 4))
(f 2)
(g 2)
```

JavaScript

```
function bar(y) {
  return function (x) {
    return x + y;
  };
}
let f = bar(2);
let g = bar(4);
console.log(f(2));
console.log(g(2));
```

7

`4 6`

8

You predicted the output correctly 🎉

The value of `(bar 2)` is a lambda function defined in an environment where `y` is bound to 2. The value of `(bar 4)` is *another* lambda function defined in an environment where `y` is bound to `4`. The two lambda functions are *different* values. So, the value of `(f 2)` is `12`, while the value of `(g 2)` is `52`.

Click here to run this program in the Stacker.

Lispy |

(Task 4 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x 1)
(deffun (f)
  (lambda (y)
    (+ x y)))
(defvar g (f))
(set! x 2)
(g 0)
```

Python

```
x = 1
def f():
    return lambda y: x + y
g = f()
x = 2
print(g(0))
```

10

`2`

11

You predicted the output correctly 🎉

`x` is bound to `1`. `g` is bound to the lambda function. `(set! x 2)` binds `x` to `2`. So, the value of `(g 0)` is the value of `(+ 2 0)`, which is `2`.

Click here to run this program in the Stacker.

Lispy |

(Task 5 of 9) What is the result of running this program?

Lispy [Run]

```
(deffun (foobar)
  (defvar n 0)
  (lambda ()
    (set! n (+ n 1))
    n))
(defvar f (foobar))
(defvar g (foobar))

(f)
(g)
(f)
```

JavaScript

```
function foobar() {
  let n = 0;
  return function () {
    n = n + 1;
    return n;
  };
}
let f = foobar();
let g = foobar();
console.log(f());
console.log(g());
console.log(f());
```

13

```
1 1 2
```

14

You predicted the output correctly 🎉

Every time `foobar` is called, it creates a *new* environment that binds `n`. So, `f` and `g` have different bindings for the `n` variable. When `f` is called the first time, it mutates its binding for the `n` variable. `g` has its own binding for the `n` variable, which still binds `n` to `0`. So, `(g)` produces `1` rather than `2`. The second call to `f` produces `2` rather than `1`.

Click here to run this program in the Stacker.

Lispy |

(Task 6 of 9)

Lispy [Run]

```
(deffun (f x y z) body)
```

JavaScript

```
function f(x, y, z) {
  return body;
}
```

is a "shorthand" for

Lispy [Run]

```
(defvar f (lambda (x y z) body))
```

JavaScript

```
let f = function (x, y, z) {
    return body;
  }
```

16

Any feedback regarding these statements? Feel free to skip this question.

(You skipped the question.)

Lispy |

(Task 7 of 9) Please rewrite this function definition as a variable definition that binds lambda function.

Lispy [Run ▶]

```
(deffun (f x) (+ x 1))
```

Scala 3

```
def f(x : Int) =
  x + 1
```

(defvar f (lambda (x) (+ x 1)))

19

Lispy |

The correct answer is

Lispy [Run ▶]

```
(defvar f (lambda (x) (+ x 1)))
```

JavaScript

```
let f = function (x) {
    return x + 1;
  }
```

A common wrong answer is

Lispy [Run ▶]

```
(defvar f (lambda (f x) (+ x 1)))
```

JavaScript

```
let f = function (f, x) {
    return x + 1;
  }
```

which makes `f` a function that takes two parameters rather than one

---

Lispy |

(Task 8 of 9) Here is a program that confused many students

Lispy [Run ▶]

```
(deffun (foo n)
  (deffun (bar)
    (set! n (+ n 1))
    n)
  bar)
(defvar f (foo 0))
(defvar g (foo 0))

(f)
(g)
(f)
```

Scala 3

```
def foo(n : Int) =
  def bar =
    n = n + 1
    n
  bar
var f = foo(0)
var g = foo(0)
println(f)
println(g)
println(f)
```

Please

1. Run this program in the stacker by clicking one of the [Run ▶] buttons above;
2. The stacker would show how this program produces its result(s);
3. Keep clicking ⏭ Next until you reach a configuration that you find particularly helpful;

4. Click 🔗Share This Configuration to get a link to your configuration;
5. Submit your link below;

22

https://smol-tutor.xyz/stacker/?syntax=Lispy&randomSeed=smol-tutor&hole=%E2%80%A2&nNext=21&program=%0A%28deffun+%28foo+n%29%0A++%28deffun+%28bar%29%0A++++%28set%21+n+%28%2B+n+1%29%29%0A+++++n%29%0A++bar%29%0A%28defvar+f+%28foo+0%29%29%0A%28defvar+g+%28foo+0%29%29%0A%0A%28f%29%0A%28g%29%0A%28f%29%0A&readOnlyMode=

23

(Task 9 of 9) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

24

''

Lispy | 🎲

Let's review what we have learned in this tutorial.

In this tutorial, we will learn about **lambda expressions**, which are expressions that create functions.

The following program illustrates how to create a function without giving it a name and then call it immediately.

Lispy [Run ▶]

```
((lambda (n)
   (+ n 1))
 2)
```

Scala 3

```
((n : Int) => n + 1)(2))
```

The function is created by a lambda expression. This function increases it parameter by one.

Lispy [Run ▶]

```
(lambda (n)
  (+ n 1))
```

Scala 3

```
(n : Int) => n + 1
```

Lispy [Run ▶]

```
(deffun (f x y z) body)
```

Scala 3

```
def f(x : Int, y : Int, z : Int) =
  body
```

is a "shorthand" for

Lispy [Run ▶]

```
(defvar f (lambda (x y z) body))
```

```
var f = (x : Int, y : Int, z : Int)
  body
```

You have finished this tutorial 🎉🎉🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781420265751